DATA ANALYSIS REPORT

EBN2 Task 2 — Data Science Capstone (D606)

Jesse Coggins

Predicting Loan Default Status Using Random Forest Classification

Western Governors University

A. Research Question

Summary of the Research Question

The research question is how well selected borrower and loan attributes distinguish default status in historical data. The study evaluates a risk-ranking model for potential underwriting review. It does not estimate the causal effect of individual financial characteristics.

Justification

Loan defaults are one of the most significant financial risks lenders face. According to the Consumer Financial Protection Bureau (CFPB), default rates have historically disproportionate impacts on both borrowers and lending institutions, contributing to financial instability (CFPB, 2022). Traditional rule-based credit scoring methods, while widely used, often fail to capture complex, non-linear interactions between borrower characteristics. A predictive machine learning model can augment these traditional approaches by identifying patterns across multiple variables simultaneously, enabling lenders to make more informed and equitable decisions. The availability of a detailed dataset provided by Western Governors University which contains over 148,000 loan records with financial, demographic, and property-related attributes. This makes this an appropriate and tractable problem for advanced data science techniques.

Context

This study applies supervised classification to historical lending outcomes. A risk score could help prioritize applications for manual review, but the test measured predictive performance only. Avoided losses, pricing improvements and fairness benefits would need separate evaluation in the intended lending setting.

The held-out ROC-AUC was 0.737 and default recall was 26.5%. The analysis measured predictive performance. It did not estimate the causal effect or statistical significance of individual features, test subgroup fairness, or measure lending losses.

Hypothesis

Objective: Evaluate whether the selected borrower and loan attributes support useful default-risk ranking on held-out data.

Chance-ranking reference: A ROC-AUC of 0.5 corresponds to chance ranking. Accuracy also depends on the class balance; a majority-class prediction is a different baseline.

Interpretation: Report ROC-AUC, precision, recall, F1 and the confusion matrix. These describe predictive performance without establishing the significance or causal effect of an individual feature.

Scope of the evaluation

The notebook evaluates a baseline model on validation data and a tuned model on test data. It does not perform a formal hypothesis test or report uncertainty intervals for predictive metrics. A test ROC-AUC above 0.5 is descriptive evidence of ranking ability in this sample; it is not, by itself, a significance test. The two model evaluations use different splits, so their difference cannot isolate the effect of tuning.

B. Data Collection

Description of Data Collected

The dataset used in this project is the WGU-provided Loan Default dataset, supplied as a flat Excel file (Loan Default Data.xlsx). The dataset contains 148,670 records and 34 columns covering a wide range of borrower and loan attributes, including demographic information, loan characteristics, financial metrics, and property details. The target variable, Status, is a binary indicator where 1 represents a defaulted loan and 0 represents a loan in good standing.

For this analysis, 14 columns were selected based on their direct relevance to the research question. The selected features include demographic and categorical variables (Gender, loan_limit, loan_purpose, business_or_commercial, Region, age, credit_type, occupancy_type) and financial/quantitative variables (loan_amount, income, Credit_Score, dtir1, property_value). The target variable Status was retained.

Data-Gathering Methodology

The data-gathering methodology employed was a secondary data collection approach using a pre-compiled institutional dataset. The data was ingested directly into a Python environment using the pandas library via the read_excel() function. No active field data collection, surveys, or web scraping was required.

Advantage: A key advantage of using a pre-compiled institutional dataset is that it eliminates the time and resource costs associated with primary data collection. The data was already validated, structured, and available in a standardized format, allowing the project to proceed directly to analysis phases without delays related to data acquisition.

Disadvantage: A disadvantage of this approach is the lack of control over data collection methodology. Because the data was compiled externally, the researcher cannot verify the exact conditions under which it was gathered, the sampling strategy used, or whether there are systematic biases in how certain borrower segments were represented. This limits the ability to fully assess external validity.

Challenges and How They Were Overcome

The primary challenge encountered during data collection was the presence of missing values across several key columns. Specifically, the following missing value counts were identified after limiting the dataset to the 14 selected columns:

- loan_limit: 3,344 missing values
- loan_purpose: 134 missing values
- age: 200 missing values
- income: some missing values
- Credit_Score: some missing values
- dtir1: some missing values

To address this challenge, rows containing any missing values were removed using the pandas dropna() function. This listwise deletion approach was chosen to preserve data integrity and avoid introducing imputation bias into the model inputs, particularly for the quantitative financial predictors. After dropping rows with missing values, the dataset retained 121,447 records — approximately 81.7% of the original records — which remained a sufficiently large and representative sample for model training and evaluation.

C. Data Extraction and Preparation

Data Extraction and Preparation Process

The data extraction and preparation process was conducted entirely within a Python-based Jupyter Notebook environment. The following steps were performed:

Step 1: Library Import and Data Ingestion

The necessary Python libraries were imported at the outset of the notebook. These included pandas for data manipulation, numpy for numerical operations, and scikit-learn for machine learning modeling and evaluation. The dataset was ingested from the Excel file using pandas read_excel(), and an initial inspection was performed using df.shape, df.info(), and df.head() to confirm the structure of the data.

```python
# Library Import
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report

# Importing Dataset
file_path = "../data/Loan Default Data.xlsx"
df = pd.read_excel(file_path)

# Display df
print(df.shape)
print(df.info())
print(df.head())
```

```
(148670, 34)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148670 entries, 0 to 148669
Data columns (total 34 columns):
 #   Column                     Non-Null Count   Dtype
---  ------                     --------------   -----
 0   ID                         148670 non-null  int64
 1   year                       148670 non-null  int64
 2   loan_limit                 145326 non-null  object
 3   Gender                     148670 non-null  object
 4   approv_in_adv              147762 non-null  object
 5   loan_type                  148670 non-null  object
 6   loan_purpose               148536 non-null  object
 7   Credit_Worthiness          148670 non-null  object
 8   open_credit                148670 non-null  object
 9   business_or_commercial     148670 non-null  object
 10  loan_amount                148670 non-null  int64
 11  rate_of_interest           112231 non-null  float64
 12  Interest_rate_spread       112031 non-null  float64
 13  Upfront_charges            109028 non-null  float64
 14  term                       148629 non-null  float64
 15  Neg_ammortization          148549 non-null  object
 16  interest_only              148670 non-null  object
 17  lump_sum_payment           148670 non-null  object
 18  property_value             133572 non-null  float64
...
3                 not_inst  69.376900  North        direct       0  42.0
4                 not_inst  91.886544  North        direct       0  39.0

[5 rows x 34 columns]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Step 2: Feature Selection

Fourteen columns were selected from the original 34-column dataset based on their conceptual relevance to the research question. The selection targeted both categorical descriptors of the borrower and loan (e.g., gender, loan purpose, region) and the core financial variables named in the hypothesis (loan_amount, income, Credit_Score, dtir1). The Status column was retained as the target variable. All other columns, including administrative identifiers and less theoretically relevant features, were dropped.

```python
# Columns to keep based on the Research Question
keep_cols = [
    # Demographics and Loan Categories
    "Gender", "loan_limit", "loan_purpose", "business_or_commercial",
    "Region", "age", "credit_type", "occupancy_type",

    # Financial/Quantitative metrics
    "loan_amount", "income", "Credit_Score", "dtir1", "property_value",

    # Target variable (1 = Default, 0 = No Default)
    "Status"
]

# Keep only the selected columns
df = df[keep_cols]

# Quick check
print(df.info())
print(f"Dataset now has {df.shape[0]} rows and {df.shape[1]} columns")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148670 entries, 0 to 148669
Data columns (total 14 columns):
 #   Column                  Non-Null Count   Dtype
---  ------                  --------------   -----
 0   Gender                  148670 non-null  object
 1   loan_limit              145326 non-null  object
 2   loan_purpose            148536 non-null  object
 3   business_or_commercial  148670 non-null  object
 4   Region                  148670 non-null  object
 5   age                     148470 non-null  object
 6   credit_type             148670 non-null  object
 7   occupancy_type          148670 non-null  object
 8   loan_amount             148670 non-null  int64
 9   income                  139520 non-null  float64
 10  Credit_Score            148670 non-null  int64
 11  dtir1                   124549 non-null  float64
 12  property_value          133572 non-null  float64
 13  Status                  148670 non-null  int64
dtypes: float64(3), int64(3), object(8)
memory usage: 15.9+ MB
None
Dataset now has 148670 rows and 14 columns
```

Step 3: Missing Value Handling

A missing value audit was conducted by calling df.isnull().sum() on the reduced feature set. Missing values were identified across several columns. The decision was made to drop all rows containing missing values using df.dropna(), resulting in a final dataset of 121,447 records. This approach was selected over imputation to avoid artificially introducing noise into critical financial variables.

```
# Check for missing values
print("Missing values per column:")
print(df.isnull().sum())

# Handle missing values by dropping them to preserve clean data without imputation bias
df = df.dropna()
print(f"Dataset shape after handling missing values: {df.shape}")
```

```
Missing values per column:
Gender                      0
loan_limit               3344
loan_purpose              134
business_or_commercial      0
Region                      0
age                       200
credit_type                 0
occupancy_type              0
loan_amount                 0
income                   9150
Credit_Score                0
dtir1                   24121
property_value          15098
Status                      0
dtype: int64
Dataset shape after handling missing values: (121447, 14)
```

Step 4: Exploratory Categorical Inspection

A loop was used to print the unique values in each object-type column. This step was critical to verifying data integrity and planning the encoding strategy. It confirmed, for example, that loan_purpose used coded labels (p1, p2, p3, p4), Region included values like South, North-East, central, and North, and Gender included Male, Female, Joint, and Sex Not Available.

```
# Loop through and print unique text variables
for column in df.columns:
    if df[column].dtype == 'object':
        print(f"Unique values in column '{column}':")
        print(df[column].unique())
        print()
```

```
Unique values in column 'Gender':
['Sex Not Available' 'Male' 'Joint' 'Female']

Unique values in column 'loan_limit':
['cf' 'ncf']

Unique values in column 'loan_purpose':
['p1' 'p4' 'p3' 'p2']

Unique values in column 'business_or_commercial':
['nob/c' 'b/c']

Unique values in column 'Region':
['south' 'North' 'central' 'North-East']

Unique values in column 'age':
['25-34' '35-44' '45-54' '55-64' '65-74' '>74' '<25']

Unique values in column 'credit_type':
['EXP' 'CRIF' 'CIB' 'EQUI']

Unique values in column 'occupancy_type':
['pr' 'sr' 'ir']
```

Step 5: One-Hot Encoding

The eight categorical columns were one-hot encoded using pandas get_dummies() with drop_first=False to preserve all category information. Boolean output columns were subsequently cast to integers (0/1). After encoding, the dataset expanded to 36 columns. A final check of the target variable distribution confirmed a significant class imbalance: 101,631 records labeled as non-default (0) versus 19,816 labeled as default (1), a ratio of approximately 5:1.

```
# Start from your selected columns
df_encoded = df.copy()

# Identify text columns to One-Hot Encode
cat_cols = [
    "Gender", "loan_limit", "loan_purpose", "business_or_commercial",
    "Region", "age", "credit_type", "occupancy_type"
]

# One-hot encode multi-category variables
df_encoded = pd.get_dummies(
    df_encoded,
    columns=cat_cols,
    drop_first=False
)

# Convert all boolean columns to integers (0/1)
bool_cols = df_encoded.select_dtypes(include=['bool']).columns
df_encoded[bool_cols] = df_encoded[bool_cols].astype(int)

# Target 'Status' distribution check
print("Target Variable Distribution:")
print(df_encoded['Status'].value_counts())
```

```
Target Variable Distribution:
Status
0    101631
1     19816
Name: count, dtype: int64
```

Step 6: Feature/Target Split and Train-Validation-Test Split

The feature matrix (X) and target vector (y) were separated by dropping the Status column from X. The data was then split into three sets using a stratified two-stage split to preserve the class distribution in each partition: 70% training (85,012 records), 15% validation (18,217 records), and 15% test (18,218 records). Stratification was applied to ensure proportional representation of both classes across all splits.

```
# Save the cleaned dataset
df_encoded.to_csv('loan_cleaned_encoded.csv', index=False)
print("Cleaned dataset saved as 'loan_cleaned_encoded.csv'")
print(f"Dataset shape: {df_encoded.shape}")
```

```
Cleaned dataset saved as 'loan_cleaned_encoded.csv'
Dataset shape: (121447, 36)
```

```
# Separate features (X) and target (y)
X = df_encoded.drop('Status', axis=1)
y = df_encoded['Status']

print(f"Features shape: {X.shape}")
print(f"Target distribution:\n{y.value_counts()}")
```

```
Features shape: (121447, 35)
Target distribution:
Status
0    101631
1     19816
Name: count, dtype: int64
```

```
# Split into train (70%) and temp (30%)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Split temp into validation (15%) and test (15%)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
)

print("Train:", X_train.shape, "Validation:", X_val.shape, "Test:", X_test.shape)
```

```
Train: (85012, 35) Validation: (18217, 35) Test: (18218, 35)
```

```
# Save features and labels for each split
X_train.to_csv("X_train.csv", index=False)
y_train.to_csv("y_train.csv", index=False)

X_val.to_csv("X_val.csv", index=False)
y_val.to_csv("y_val.csv", index=False)

X_test.to_csv("X_test.csv", index=False)
y_test.to_csv("y_test.csv", index=False)

print("Datasets saved as CSV files:")
print("X_train.csv, y_train.csv, X_val.csv, y_val.csv, X_test.csv, y_test.csv")
```

```
Datasets saved as CSV files:
X_train.csv, y_train.csv, X_val.csv, y_val.csv, X_test.csv, y_test.csv
```

Tools and Techniques

The primary tools used were Python 3, Jupyter Notebook, and the following libraries:

- pandas: Data ingestion, filtering, missing value handling, and one-hot encoding
- numpy: Numerical array operations
- scikit-learn: Model building, hyperparameter tuning, and evaluation metrics

Advantage: Python with pandas and scikit-learn is an industry-standard toolkit for data science workflows. It offers exceptional flexibility, well-documented APIs, and seamless integration between data preparation and modeling steps. The scikit-learn library in particular provides a consistent interface for model training, evaluation, and tuning.

Disadvantage: A disadvantage of this approach is that Jupyter Notebooks are not inherently designed for production deployment. The lack of version control integration and the notebook's stateful execution model can introduce reproducibility issues if cells are run out of order.

Encoding and Scaling Justification

Categorical variables were transformed using one-hot encoding to convert non-numeric attributes into machine-readable binary indicator variables. This technique allows categorical data to be incorporated into the Random Forest model without introducing ordinal bias — that is, without implying that one category is numerically greater than another. Unlike distance-based models such as k-nearest neighbors or support vector machines, Random Forest does not require feature scaling or normalization because decision trees split on thresholds rather than relying on Euclidean distance.

Advantage: One-hot encoding preserves categorical distinctions without imposing any magnitude or ordering relationship between categories, ensuring that the model treats each level independently.

Disadvantage: A disadvantage of one-hot encoding is that it increases dimensionality when categorical variables contain many unique levels, which can slow training time and introduce sparsity into the feature space.

D. Analysis

Analysis Technique: Random Forest Classification

The analysis technique selected for this project is the Random Forest Classifier, an ensemble machine learning method that constructs a multitude of decision trees during training and outputs the class that is the mode of the classes predicted by individual trees (Breiman, 2001). This technique was chosen because the research question requires predicting a binary categorical outcome (Default vs. Non-Default) from a mixture of numerical and encoded categorical features — a task well-suited to classification methods.

Justification for Technique Selection

Random Forest was selected over simpler alternatives such as Logistic Regression or a single Decision Tree for three primary reasons. First, as an ensemble method, it reduces the variance of predictions through aggregation, mitigating the overfitting that commonly affects individual decision trees. Second, Random Forest can model complex, non-linear interactions between variables — for example, the combined effect of a high debt-to-income ratio and a low credit score — that linear models cannot capture. Third, Random Forest provides native feature importance scores, allowing for post-hoc interpretation of which financial variables most strongly influence default predictions.

Advantage: The key advantage of Random Forest is its robustness to overfitting and its ability to handle high-dimensional feature spaces with mixed variable types without requiring feature scaling. The use of class_weight='balanced' directly addresses the class imbalance present in this dataset by assigning higher misclassification costs to the minority class (defaults).

Disadvantage: A notable disadvantage of Random Forest is its computational intensity. Training 500 trees over a dataset of 85,012 records with 35 features — especially during the GridSearchCV hyperparameter tuning phase (240 total fits across 5-fold cross-validation) — requires significant processing time and memory. The model is also less interpretable than simpler models such as Logistic Regression.

Phase 1: Base Model Training and Validation

A baseline Random Forest model was trained using 300 estimators (n_estimators=300) with class_weight='balanced' and random_state=42 for reproducibility. The model was evaluated on the held-out validation set (18,217 records). The resulting metrics were:

```
Accuracy : 0.853
Precision: 0.758
Recall   : 0.148
F1-Score : 0.248
ROC-AUC  : 0.733
```

The base model achieved high accuracy but exhibited a very low recall (0.148), meaning it correctly identified only 14.8% of actual defaulters. The confusion matrix showed 2,531 false negatives — borrowers who defaulted but were predicted as non-defaulters. While the precision of 0.758 was high, the model's ability to surface defaults in practice was limited, motivating hyperparameter optimization.

Phase 2: Hyperparameter Tuning via GridSearchCV

A 5-fold cross-validated grid search (GridSearchCV) was conducted over the following hyperparameter space:

- n_estimators: [200, 500]
- max_depth: [None, 10, 20]
- min_samples_split: [2, 5]
- min_samples_leaf: [1, 2]
- max_features: ['sqrt', 'log2']

A total of 48 candidate configurations were evaluated, resulting in 240 model fits. The scoring metric was ROC-AUC, which is appropriate for imbalanced classification tasks because it evaluates the model's ability to distinguish between classes across all classification thresholds.

The optimal configuration identified was:

```
Best Parameters: n_estimators=500, max_depth=None, max_features='sqrt',
                 min_samples_leaf=2, min_samples_split=2
Best CV ROC-AUC: 0.740
```

Phase 3: Final Model Evaluation on Test Set

The tuned model was trained on the full training set (85,012 records) using the best hyperparameters and evaluated on the untouched test set (18,218 records). Final results were:

```
Accuracy : 0.847
Precision: 0.570
Recall   : 0.265
F1-Score : 0.362
ROC-AUC  : 0.737
```

The following table summarizes the performance comparison between the base and tuned models:

Metric	Base Model (Val Set)	Tuned Model (Test Set)	Notes
Accuracy	0.853	0.847	Different split
Precision	0.758	0.570	Different split
Recall	0.148	0.265	Different split
F1-Score	0.248	0.362	Different split
ROC-AUC	0.733	0.737	Different split

The following confusion matrix presents the final tuned model's test set predictions:

	Predicted: No Default (0)	Predicted: Default (1)	Total
Actual: No Default (0)	14,652 (TN)	593 (FP)	15,245
Actual: Default (1)	2,186 (FN)	787 (TP)	2,973
Total	16,838	1,380	18,218

The tuned model flagged 787 defaults and missed 2,186. Its test recall was 26.5%, versus 14.8% for the baseline on a different validation split. These evaluations cannot isolate the effect of tuning. Test ROC-AUC was 0.737; no formal significance test was performed.

Overfitting Assessment and Bias-Variance Considerations

The saved notebook does not calculate training-set accuracy. It therefore does not establish similar training and test accuracy or rule out overfitting through that comparison. The untouched test set provides a held-out performance estimate, while cross-validation supports model selection. External validation and uncertainty estimates would strengthen the assessment.

Advantage: A key advantage of Random Forest classification is its ability to capture complex, non-linear relationships between features while maintaining robustness against overfitting, making it well-suited for high-dimensional financial datasets.

Disadvantage: A disadvantage is reduced interpretability compared to simpler linear models. The internal structure of hundreds of decision trees is not easily explained to stakeholders without the use of additional interpretability tools such as SHAP (SHapley Additive exPlanations) values.

E. Data Summary and Implications

Results in the Context of the Research Question

The tuned model achieved test ROC-AUC of 0.737 and identified 787 of 2,973 defaults. This describes ranking and classification on the held-out sample. The notebook does not test individual-feature significance, estimate causal effects, or provide feature-importance results that establish which financial attribute drove performance.

The score could support prioritizing manual underwriting review. Whether this improves loss rates, pricing or operational efficiency would require a separate comparison in the intended lending setting. Those outcomes were not measured here.

However, the results also reveal the inherent difficulty of predicting rare credit events. The 5:1 class imbalance between non-defaults and defaults created a challenging learning environment. Despite the use of class_weight='balanced' to penalize default misclassifications more heavily, the model's recall for the default class remained at 26.5%, meaning nearly three-quarters of defaulters in the test set were not flagged by the model. This is a known challenge in credit risk modeling and is consistent with findings reported in academic literature on financial default prediction (Lessmann et al., 2015).

Limitation of the Analysis

A key limitation of this analysis is reliance on a single historical dataset without external validation. The model was trained and evaluated on data from one institutional source, which limits generalizability to other lending contexts or time periods. Additionally, while predictive accuracy was strong overall, the model does not inherently address fairness or potential bias across demographic groups — the dataset contains sensitive variables such as Gender, Region, and age, and differential error rates across these groups were not evaluated in this study. Finally, the decision to handle missing values through listwise deletion may have introduced subtle selection bias if the removed records were not missing at random.

Recommended Course of Action

A supervised pilot could evaluate the tuned model as a review-prioritization tool after external validation and subgroup checks. Human underwriters would retain decision responsibility. Threshold analysis should quantify missed defaults and review workload against an agreed cost structure. The tested threshold caught 26.5% of defaults; no deployed financial benefit was measured.

Future Directions for Study

Two promising directions are proposed for future study of this dataset:

Future Direction (1) — Gradient Boosting Comparison and Threshold Optimization: Future research should compare Random Forest performance against gradient boosting models such as XGBoost or LightGBM, which are known to achieve strong performance on imbalanced tabular classification tasks. Additionally, a systematic threshold-tuning analysis using precision-recall curves could identify the optimal classification cutoff aligned with the lender's cost structure for false positives versus false negatives. This direction directly addresses model performance limitations identified in the current analysis.

Future Direction (2) - Explainability and Fairness: Add interpretation tools such as SHAP to inspect how inputs influence predictions. Measure error rates and calibration across Gender, Region and age groups. These checks would inform further model and policy review; they do not by themselves establish fair outcomes or regulatory compliance (Barocas & Hardt, 2019).

F. Sources

Barocas, S., & Hardt, M. (2019). Fairness and machine learning: Limitations and opportunities. fairmlbook.org. https://fairmlbook.org

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Consumer Financial Protection Bureau. (2022). Consumer credit trends. CFPB. https://www.consumerfinance.gov/data-research/consumer-credit-trends/

Lessmann, S., Baesens, B., Seow, H.-V., & Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. European Journal of Operational Research, 247(1), 124–136. https://doi.org/10.1016/j.ejor.2015.05.030

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Western Governors University. (2024). Loan default dataset [Data file]. WGU D606 Data Science Capstone course materials.